

Group 3A Expense Tracker API: Work Division for 13 Members

The project is a Node.js and Express REST API for tracking expenses. It has about 650 lines of code, tests and README, plus a long build guide. We split it into **13 parts**. Each member owns one part and pushes it to GitHub as their own commit(s).

The parts are balanced by effort, at roughly 40 to 65 lines of code each. Part 13 is documentation only, and most of its text already exists.

1. Overview

#	Part	Files owned	Approx. lines
1	Project setup	<code>package.json</code> , <code>package-lock.json</code> , <code>.gitignore</code> , <code>index.js</code>	37
2	Data layer	<code>src/data/categories.js</code> , <code>src/data/db.js</code>	43
3	Seed script, error handling, server entry	<code>src/data/seed.js</code> , <code>src/middleware/errorHandlers.js</code> , <code>src/server.js</code>	54
4	Input validation	<code>src/middleware/validateExpense.js</code>	68
5	Controller foundation	<code>src/controllers/expenseController.js</code> (helpers + categories)	50
6	Create, read one, delete	<code>expenseController.js</code> (3 functions)	41
7	List with filters, search, sort, paging	<code>expenseController.js</code> (<code>getExpenses</code>)	60
8	Update and summary	<code>expenseController.js</code> (2 functions)	49
9	Routes and app assembly	<code>src/routes/expenseRoutes.js</code> , <code>src/app.js</code>	56
10	Tests A: setup, health, POST	<code>tests/expenses.test.js</code> (lines 1-56)	56
11	Tests B: GET (list, filters, by id)	<code>tests/expenses.test.js</code> (lines 58-96)	39
12	Tests C: PUT, PATCH, DELETE, summary, 404	<code>tests/expenses.test.js</code> (lines 98-144)	47
13	Documentation and final check	<code>README.md</code> , <code>BUILD-GUIDE.md</code> , <code>PROJECT-DIVISION.md</code>	docs

2. Merge order

The parts depend on each other, so **merge them in numeric order (1 to 13)**. You can write your part any time. Push and merge it only after the part before yours has been merged.

- Parts **5, 6, 7 and 8** all add code to the same file, `expenseController.js`. Each one adds its own functions and its own name in `module.exports`. Pull the latest `main` before you start and before you push.
- Parts **10, 11 and 12** all add tests to `tests/expenses.test.js`. Each one appends its own tests at the end of the file.
- The API only runs end to end after **Part 9** is merged, and the tests only pass after Part 12. That is normal. Check your own part by reading the code, or by running it once the earlier parts are in.

3. What each member does

Part 1: Project setup

- Files: `package.json`, `package-lock.json`, `.gitignore`, `index.js`
- Create the Node project (`npm init`) and install `express`, `cors` and `morgan`.
- Install `nodemon` and `supertest` as dev dependencies.
- Add the npm scripts: `start`, `dev`, `test`, `seed`.
- Add `.gitignore` (it ignores `node_modules/` and `.env`). Add the original "Hello World" `index.js`.
- Create the empty folder layout: `src/controllers`, `src/data`, `src/middleware`, `src/routes`, `tests`, `data`.

Part 2: Data layer

- Files: `src/data/categories.js`, `src/data/db.js`
- `categories.js` lists the 9 allowed categories: Food, Transport, Housing, Utilities, Health, Education, Entertainment, Shopping and Other.
- `db.js` stores data in a JSON file. It has `ensureFile`, `readAll` and `writeAll`. The path can be changed with the `DATA_FILE` environment variable.
- Cleanup to do in this part: remove the unused `isUtf8` import, and make the empty-file fallback in `readAll` the string `[]` instead of an array.

Part 3: Seed script, error handling, server entry

- Files: `src/data/seed.js`, `src/middleware/errorHandlers.js`, `src/server.js`
- `seed.js` loads 8 sample expenses (`npm run seed`).
- Bugs to fix in this part: `new Date().toISOString` is missing its `()`, so `createdAt` and `updatedAt` end up as the text `'now'`. The unused `require('express')` can go.
- `errorHandlers.js` has the `notFound` handler (404) and the `errorHandler`. The error handler turns bad JSON into a clean 400 response.
- `server.js` reads `PORT` (default 3000) and starts the app.

Part 4: Input validation

- File: `src/middleware/validateExpense.js`
- `isValidDate` checks the `YYYY-MM-DD` format and rejects impossible dates such as `2026-02-30`.
- `checkFields` validates title (required, at most 100 characters), amount (a number greater than 0), category (must be in the list), date, and an optional description.
- `validateExpense({ partial })` is the middleware. `partial` is true for PATCH, where fields are optional. It returns `400` with a list of errors.

Part 5: Controller foundation

- File: `src/controllers/expenseController.js` (create the file)
- Imports (`crypto`, `db`, `categories`, `isValidDate`) and the helpers:
 - `round2`: round money to 2 decimals.
 - `pickFields`: copy only the allowed fields, so a client cannot set `id` or `createdAt`.
 - `applyFilters`: shared category and date-range filter.
- `getCategories` (GET `/api/categories`).
- Start `module.exports` with `getCategories`. Later parts add their own names.

Part 6: Create, read one, delete

- File: `src/controllers/expenseController.js`
- `createExpense`: build the expense with a UUID and timestamps, save it, and return `201`.
- `getExpenseById`: return the expense, or `404` if it does not exist.
- `deleteExpense`: remove the expense, or return `404` if it does not exist.
- Add all three names to `module.exports`.

Part 7: List with filters, search, sort and paging

- File: `src/controllers/expenseController.js`
- `getExpenses` is the biggest function:
 - uses `applyFilters` for category and dates,
 - filters by `minAmount` and `maxAmount`,
 - searches title and description with `search`,
 - sorts by `sort` (a leading - means descending),
 - pages the results with `page` and `limit`,
 - returns `totalItems` and `totalAmount`.
- Add `getExpenses` to `module.exports`.

Part 8: Update and summary

- File: `src/controllers/expenseController.js`
- `updateExpense` handles both PUT (full replace, so an omitted description is cleared) and PATCH (partial update). `id` and `createdAt` are kept, and `updatedAt` is refreshed.
- `getSummary` gives the total, the count, the average, and totals per category. It accepts optional filters.
- Add both names to `module.exports`.

Part 9: Routes and app assembly

- Files: `src/routes/expenseRoutes.js`, `src/app.js`
- `expenseRoutes.js` maps each method and URL to a controller function. `/summary` must be declared before `/:id`. PUT uses `validateExpense()` and PATCH uses `validateExpense({ partial: true })`.
- `app.js` sets up `cors`, `express.json`, and `morgan` (off when `NODE_ENV=test`). It defines the `/` welcome route, `/api/health`, `/api/categories` and `/api/expenses`. It ends with `notFound` and `errorHandler`.
- After this part merges, run `npm start` and open `http://localhost:3000` to check.

Part 10: Tests A (setup, health, POST)

- File: `tests/expenses.test.js` (create the file, lines 1-56)
- Test setup: temp data file, `beforeEach` reset, and cleanup afterwards.
- Tests: `/api/health`, POST creates an expense, POST rejects invalid data, POST rejects malformed JSON, and POST ignores fields the client may not set.

Part 11: Tests B (GET)

- File: `tests/expenses.test.js` (append lines 58-96)
- Tests: list with totals (newest first), filters (category, date range, amount), an invalid filter returns 400, and GET by id returns 200 or 404.

Part 12: Tests C (PUT, PATCH, DELETE, summary, 404)

- File: `tests/expenses.test.js` (append lines 98-144)
- Tests: PUT replaces, PUT needs all fields, PATCH changes only what was sent, DELETE removes, summary totals by category, and unknown routes return 404.
- After merging, run `npm test`. All tests should pass.

Part 13: Documentation and final check

- Files: `README.md`, `BUILD-GUIDE.md`, `PROJECT-DIVISION.md`
- Commit the README (features, setup, project structure, API table, examples, configuration).
- Commit the step-by-step `BUILD-GUIDE.md` and this division document.

- Do the final check on a fresh clone: `npm install`, `npm run seed`, `npm start`, `npm test`. Fix any broken docs.

4. Git and GitHub steps

One person (the team lead) sets up the repository, once:

```
git init
git branch -M main
git remote add origin https://github.com/<team-or-user>/<repo-name>.git
```

Then create the empty repo on GitHub and add all 13 members as collaborators (Settings, Collaborators). Push the first commit, which is Part 1.

Every member:

```
git clone https://github.com/<team-or-user>/<repo-name>.git
cd <repo-name>
git checkout -b part-05-controller-foundation # your own branch, named after your part
# ...write or copy your files...
git add <your files only>
git commit -m "Part 5: controller foundation (helpers and getCategoryes)"
git pull --rebase origin main # get the parts merged before yours
git push -u origin part-05-controller-foundation
```

Then open a Pull Request into `main`. The team lead merges the pull requests **in order, 1 to 13**.

- Commit only your own files, so the history clearly shows who did what.
- Write a clear commit message that starts with your part number, for example **Part 7: list expenses with filters, sort and paging**.
- If you get a merge conflict in `expenseController.js` or `expenses.test.js`, keep both sides. Everyone's functions and tests should stay in the file.

5. Known issues to fix while committing

Part	File	Issue
2	<code>src/data/db.js</code>	Unused <code>isUtf8</code> import. <code>JSON.parse</code> is given an array as its fallback instead of the string <code>[]</code> .
3	<code>src/data/seed.js</code>	<code>new Date().toISOString</code> is not called. <code>createdAt</code> and <code>updatedAt</code> are stored as the text <code>'now'</code> . Unused <code>express</code> import.